

TECHNICAL DOCUMENT 3: DOCKER, CI/CD PIPELINE & GIT WORKFLOW GUIDE

Movie Review & Finder Web Application

This document provides a comprehensive, step-by-step guide to setting up and running Docker containers, deploying CI/CD pipelines inside GitHub, and applying high-precision Git workflows.

Table of Contents

1. Docker Containerization
 - Production Container ([Dockerfile](#))
 - CI/CD Testing Container ([Dockerfile.cicd](#))
 - Local Docker Commands
 2. CI/CD Automated Pipeline Configuration
 - Unified Automation Script ([run-pipeline.sh](#))
 - GitHub Actions Workflow Configuration
 - Secret Management inside GitHub
 3. Professional Git Version Control & Functions
 - Essential Git Command Reference
 - Branching & Pull Request Workflows
 - Conflict Resolution Guide
-

1. Docker Containerization

We provide two separate Docker configurations tailored for different phases of the software delivery lifecycle:

Production Container ([Dockerfile](#))

Optimized for lightweight, multi-stage deployments to hosting services like Google Cloud Run or AWS ECS. It excludes testing libraries (like Playwright), minimizing container size and cold-start latency.

Production Configuration:

- **Base Image:** `node:20-alpine` (extremely small footprint, ~150MB).
 - **Production Flags:** Installs dependencies using `npm ci --only=production`.
 - **Port binding:** Binds to port `3000` on host interface `0.0.0.0` for optimal request routing.
-

CI/CD Testing Container ([Dockerfile.cicd](#))

A self-contained testing environment configured to run browser automation. It has Node.js and all system dependencies for Chromium, Firefox, and WebKit browsers pre-installed.

Pipeline Configuration ([Dockerfile.cicd](#)):

```
# Use the official Microsoft Playwright image as the base
FROM mcr.microsoft.com/playwright:v1.49.0-noble

# Set working directory inside the container
WORKDIR /app

# Set pipeline-friendly environment variables
ENV PORT=3000
ENV NODE_ENV=test
ENV CI=true

# Copy dependency configurations and perform clean install
COPY package*.json ./
RUN npm ci

# Copy the rest of the workspace files
COPY . .

# Ensure our custom pipeline script has executable permissions
RUN chmod +x /app/run-pipeline.sh

# Run the automated CI/CD pipeline script by default
CMD ["/app/run-pipeline.sh"]
```

Local Docker Commands

You can build and run both configurations locally to ensure consistency between your development workspace and production servers.

Running the Production Container:

```
# 1. Build production container
docker build -t movie-app-prod -f Dockerfile .

# 2. Run container locally mapping port 3000
docker run -p 3000:3000 --env-file .env movie-app-prod
```

Running the CI/CD Pipeline Container locally:

```
# 1. Build testing container
docker build -t movie-app-cicd -f Dockerfile.cicd .

# 2. Execute CI/CD pipeline (automatically runs install, build, start, and tests)
docker run -it -p 3000:3000 movie-app-cicd
```

2. CI/CD Automated Pipeline Configuration

Unified Automation Script (`run-pipeline.sh`)

Rather than maintaining separate configurations across different platforms, our environment relies on a single **shell script** (`run-pipeline.sh`). This script handles the pipeline logic, making it easy to run and debug both locally and in CI:

```
#!/bin/bash
set -e # Exit immediately if any command fails

# Setup cleanup on script exit to kill background servers safely
SERVER_PID=""
cleanup() {
  if [ -n "$SERVER_PID" ]; then
    echo "Stopping background server (PID: $SERVER_PID)..."
    kill "$SERVER_PID" 2>/dev/null || true
  fi
}
trap cleanup EXIT

echo "📦 Step 1: Installing dependencies..."
npm ci

echo "🔧 Step 2: Building application assets & server..."
npm run build

echo "🌐 Step 3: Starting full-stack server..."
npm start &
SERVER_PID=$!

echo "⌚ Step 4: Waiting for server to be ready on port 3000..."
max_attempts=30; attempt=1; server_ready=0
while [ $attempt -le $max_attempts ]; do
  if curl -s http://localhost:3000/api/health > /dev/null; then
    echo "✅ Server is online and healthy!"
    server_ready=1
    break
  fi
  sleep 1
  attempt=$((attempt + 1))
done

if [ $server_ready -ne 1 ]; then
  echo "❌ Error: Server failed to start on port 3000 within 30 seconds."
  exit 1
fi

echo "🧪 Step 5: Running Playwright E2E tests..."
export CI=true
npx playwright test
```

GitHub Actions Workflow Configuration (`.github/workflows/deploy.yml`)

The workflow file executes automatically on **every single Git push** or **Pull Request** targeting the main branches.

```
name: CI/CD Pipeline

on:
  push:
    branches: [ "main", "master" ]
  pull_request:
    branches: [ "main", "master" ]

jobs:
  test-and-lint:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code Repository
        uses: actions/checkout@v4

      - name: Setup Node.js Environment
        uses: actions/setup-node@v4
        with:
          node-dir: 20
          cache: 'npm'

      - name: Install Project Dependencies
        run: npm ci

      - name: Run Linter Validation
        run: npm run lint

      - name: Run Unit Test Suite (Vitest)
        run: npm run test

      - name: Install Playwright Browsers
        run: npx playwright install --with-deps chromium

      - name: Run E2E Test Suite (Playwright)
        run: npm run test:e2e
```

Secret Management inside GitHub

Our configuration checks whether Google Cloud configurations exist before triggering production image pushes. To set up automatic deployments:

1. Navigate to your GitHub Repository -> **Settings** -> **Secrets and variables** -> **Actions**.
2. Add the following secrets:
 - `GCP_PROJECT_ID`: Your Google Cloud Project identifier.

- `GCP_SA_KEY`: Your service account JSON credentials.
- `GCP_WORKLOAD_IDENTITY_PROVIDER`: Optional, modern OIDC identity provider string.

3. Professional Git Version Control & Functions

Maintaining clean branch logic and committing frequently is essential for high-velocity software engineering.



Essential Git Command Reference

Starting and Synchronizing:

- **Clone Repository:** downloads project to local systems:

```
git clone <url>
```

- **Fetch Remote Updates:** pulls updates without modifying your local files:

```
git fetch origin
```

- **Pull Down Updates:** fetches and merges remote updates into your active branch:

```
git pull origin main
```

Development & Tracking:

- **Check Working Status:** lists edited, deleted, and untracked files:

```
git status
```

- **Review File Diffs:** shows exactly what lines changed:

```
git diff
```

- **Stage Modifications:** adds files to the staging area:

```
git add .
```

- **Commit Changes:** saves staged modifications to your local history with a clear commit message:

```
git commit -m "feat: added login automation selectors and self-healing tests"
```

Publishing and History:

- **Push Commits:** uploads your local history to the remote repository (GitHub):

```
git push origin feature/my-feature-name
```

- **Review Commit Logs:** lists commit logs in reverse chronological order:

```
git log --oneline -n 10
```

Branching & Pull Request Workflows

To protect code quality in collaborative projects, avoid committing directly to the main production branch. Always follow a **Feature-Branch workflow**:

1. Create an Isolated Branch:

```
# Create and switch to a new branch  
git checkout -b feat/add-search-filters
```

2. Develop & Test Locally: Make your changes, then verify they pass all checks:

```
npm run lint  
npm run test  
npm run test:e2e
```

3. Push Branch & Create Pull Request (PR):

```
git push origin feat/add-search-filters
```

Go to GitHub and click "Compare & pull request" to open your PR. This triggers the GitHub Actions workflow, running your automated tests before your code is merged.

Conflict Resolution Guide

Merge conflicts occur when multiple developers edit the same lines of code. To resolve conflicts safely:

1. Update your local branch with the latest changes from main:

```
git checkout main
git pull origin main
git checkout feat/add-search-filters
git merge main
```

2. Identify and Fix Conflict Blocks: Open the conflicted files in your editor. Locate the merge markers:

```
<<<<<<< HEAD
const databasePort = 3000;
=====
const databasePort = process.env.PORT || 3000;
>>>>>>> main
```

Choose which version to keep, delete the conflict markers (<<<<<<<, =====, >>>>>>>), and save the file.

3. Verify, Stage, and Commit:

```
npm run build
git add .
git commit -m "chore: resolved merge conflict with main"
git push origin feat/add-search-filters
```